# Predicting Flight Disruption Using Spark: Integrating Airline Operations Data with Semi-Structured Weather Data

**Team Members:**
Atai Kamchybekov
Paul Nguyen
Nicholas Sopcic
Ilian Zalomai

**Project Repository:**
https://github.com/ilian-zalomai/Final-Project-Big-Data-Analytics-MLOps-using-Hive-Spark/tree/main

## Introduction

Our dataset is built around a structured U.S. flights dataset from Kaggle, supported by separate reference tables for airports and airlines. To meet the semi-structured data requirement, we also pulled daily weather data in JSON format from the Open-Meteo API, parsed it into a clean table called weather_daily, and engineered features such as temp_avg, precip_sum, and wind_gust_max along with simple weather flags. We then joined weather to flights using (ORIGIN_AIRPORT, FL_DATE) so we could analyze and model how local weather conditions relate to flight delays or cancellations.

All work was completed in Databricks using Apache Spark. We used Spark SQL (Hive) and PySpark DataFrames for ingestion, cleaning, transformations, and joins, and Spark MLlib for training and evaluating machine learning models (including cross-validation for tuning). We also used MLflow to track experiments by logging model parameters, evaluation metrics, and saved model artifacts, so the modeling results are reproducible and easy to compare.

Due to the size of our models and the cache limitations of the free Databricks software, **we have built our workflow to run Apache Spark in a local Python 3.13 environment.** Google Colab was not used as the service slows down significantly when a Spark session was initiated.

## Phase 1 — Problem & Goal (Business framing)

In this project, we focus on flight disruption across two outcomes with direct operational impact:

1. Classification: predict whether a flight will be cancelled (CANCELLED = 0/1).
2. Regression: predict the arrival delay in minutes (using an arrival-delay target that can be capped to reduce extreme outliers).

The decision we are supporting is basically: when and where should airlines/airports expect higher disruption risk, and how can they plan staffing, gates, and communication to passengers more proactively. This matters because cancellations and large delays create real operational costs (rebooking, crew rescheduling, gate and runway congestion) and hurt customer experience (missed connections, longer travel times, complaints). Weather is a key external factor, so we integrate

semi-structured weather data to check whether it improves prediction and also to generate insights that are easy to act on.

We define success in two layers. From a modeling perspective, a "good" result means our Spark ML models beat a simple baseline: for classification, higher AUC and F1 (and reasonable precision/recall); for regression, lower RMSE/MAE compared to a baseline predictor. We also expect the models to be evaluated fairly using train/test splits (and cross-validation for tuning). From a business perspective, success means we can extract a few clear and defensible insights (for example, disruption patterns by airport/season and how weather flags change risk) and turn them into actionable recommendations that a real operations team could use.

**Phase 2 — Data Sources & Ingestion (Big data processing)**

For Phase 2, we ingested three structured CSV files from the flights dataset (flights.csv, airports.csv, and airlines.csv) into Databricks and loaded them using Spark so they can be queried both with Spark SQL (Hive) and PySpark DataFrames. After loading, we did basic checks like row counts, schema/type checks, and simple data quality checks (for example, looking for duplicates and reviewing missing values). We also created a proper flight date column (FL_DATE) from the YEAR, MONTH, and DAY fields, because we needed a consistent date key for analysis, time-based splits, and the weather join.

**To satisfy the semi-structured data requirement**, we used daily weather data pulled from the Open-Meteo API in JSON format. We parsed that JSON into a clean daily table called weather_daily, and engineered a small set of weather features that are easy to interpret, such as temp_avg, precip_sum, and wind_gust_max, plus simple indicator flags like precip_flag and severe_weather_flag. Then we integrated weather with the flights data by joining on (ORIGIN_AIRPORT, FL_DATE), so each flight record is enriched with the weather conditions at its origin airport on that day (Exhibit 1).

**During preprocessing**, we focused on making the dataset usable for both EDA and modeling. That mainly meant standardizing data types, handling missing values in a consistent way (for example, filling weather flags with 0 when weather is missing, and treating missing numeric fields carefully), and selecting columns that make sense as features. We also avoided using "leakage" columns that directly explain delay reasons (like detailed delay-cause fields) because those can make the model look unrealistically strong. Finally, we saved the cleaned and joined outputs in a storage-friendly format (like Parquet) so later phases can run from the same reproducible "gold" dataset without redoing ingestion every time.

**Phase 3 — EDA & Key Insights**

We explored the joined flights_gold dataset to understand delay/cancellation patterns, check data quality, and identify which features are useful for modeling and business recommendations.

Delays have strong seasonality. Delay rate is highest in early summer and late winter. June has the highest delay rate (22.5%) and February is very close (22.4%). In contrast, September–October are the best months, with October the lowest (11.6%) and September (12.4%). This supports

treating calendar features (like ‘month’) as meaningful predictors and also suggests operational planning should be more cautious in peak-risk months. Mean delay minutes follow the same seasonal pattern. Average arrival delay is highest around peak months (especially mid-year) and dips below zero in September–October (meaning flights arrive slightly early on average in those months). This supports the idea that the “bad months” aren’t only more frequently delayed — they are also delayed by more minutes on average.

Day-of-week differences are real. Delays are most common on Thursday (19.3%) and Monday (19.1%), while Saturday (15.3%) is the best day. This matches a weekly travel rhythm and suggests staffing and buffer planning can be adjusted by weekdays.

Weather flags clearly increase delay risk. Weather enrichment provides an actionable signal. With no severe weather, precipitation alone raises delay rate from 15.2% (no precip) to 20.4% (precip). Under severe weather, delay rate rises further to 25.5% when precipitation is present, compared to 16.8% when there is severe weather but no precipitation. This is a strong justification for including the weather features/flags in modeling and operational alerts.

Airlines differ meaningfully in disruption patterns. Airline-level summary stats show meaningful variation in delay rate, mean delay minutes, and cancellation rate. For example, Spirit has a high delay rate (28.3%) and high mean arrival delay (14.47 min), while Hawaiian has a low delay rate (10.5%) and very low cancellation rate (0.2%). This confirms that airline identity should be included as a categorical feature and gives a business angle for performance benchmarking.

**Phase 4: Modeling (Spark MLlib)**

**Modeling Methodology**

To build a reliable predictive foundation, the flight dataset was carefully prepared to ensure the model only learns from information available before a flight departs, eliminating any risk of the model "cheating" by training on outcomes it wouldn't know in practice. Key variables such as departure time and seasonality were mathematically re-encoded to reflect their natural patterns, skewed distributions were normalized to improve prediction stability, and gaps in weather data were filled using conservative estimates derived strictly from historical training data.

Two predictive models were then developed and evaluated: one that flags individual flights as likely delayed or on-time, and one that estimates how many minutes late a flight is expected to arrive. Four industry-standard machine learning algorithms were tested across both tasks, ranging from interpretable baseline models to more sophisticated gradient boosting techniques. All experiments were logged systematically to ensure full reproducibility and auditability of results.

**Model Comparison & Selection**

Gradient Boosting emerged as the clear choice for deployment across both prediction tasks, and it should form the foundation of any operational implementation going forward.

This recommendation is supported by consistent outperformance across every measure that matters in an operational context. For the delay classification task, flagging flights likely to arrive more than 15 minutes late, Gradient Boosting achieved the highest accuracy in correctly identifying at-risk flights (AUC-ROC: 0.677) and the strongest balance of catching true delays without over-alerting operations teams (Precision: 79%, Recall: 82%). For the delay severity task of estimating how many minutes late a flight will arrive Gradient Boosting again led the field with the lowest average prediction error (~19 minutes MAE, ~30.6 minutes RMSE), meaning its estimates are consistently closest to what happens on the ground.

Each of the three alternatives had meaningful limitations. Logistic and Linear Regression, while highly interpretable, produced noticeably weaker accuracy because delay patterns are inherently non-linear. A flight at 6am on a calm Tuesday behaves very differently from the same route on a stormy Friday evening, and simpler models struggle to capture that complexity. Random Forest, a more sophisticated approach, matched Gradient Boosting on recall but fell short on precision, meaning it would generate more false alarms and erode operational trust over time. Gradient Boosting's architecture (which builds predictions iteratively, each round learning from prior errors) proved best suited to the layered complexity of flight delay patterns driven by weather, airline, route, and timing interactions simultaneously.

**Phase 5 — MLOps and Reproducibility (Feature Pipeline + MLflow Tracking)**

To keep our modeling work consistent and reproducible, we implemented a structured preprocessing and feature engineering pipeline before training models. This included cyclical encoding for time-based variables (month, day-of-week, scheduled departure time), median imputation for weather fields (computed on the training split only to avoid leakage), and stabilizing transformations such as capping extreme arrival delays (99th percentile) and using log1p transforms for skewed variables like distance. After these transformations, we rebuilt a single final feature set that is reused across models so comparisons are fair.

For experiment tracking, we used MLflow to log model runs and compare results side-by-side for both regression and classification. Based on the tracked runs, the best regression model for ARRIVAL_DELAY_CAPPED was GBTRegressor (RMSE 30.5897, MAE 19.0757, $R^2$ 0.0743). For the classification task, GBTClassifier achieved the best AUC-ROC (0.6774) compared with Logistic Regression (0.6655) and Random Forest (0.6104). Overall, the combination of a consistent feature pipeline and MLflow run tracking made our workflow easier to reproduce and our final model choice easier to justify.

**Phase 6: Recommendations & Next Steps**

**Business Implications & Operational Impact**

These models are ready to create measurable value today, across four operational areas where early, accurate prediction directly translates to cost savings and a better passenger experience.
**Proactive Crew & Gate Management.** With the model correctly identifying 4 in 5 delayed flights before departure (82% recall), ground operations teams gain a reliable early warning system. Airlines can use pre-departure delay flags to reposition aircraft, adjust crew assignments, and

reallocate gates before a single-flight disruption cascades into a network-wide problem, directly attacking late aircraft delay, the second-largest delay driver in the dataset.

**Weather-Triggered Contingency Protocols.** Weather drove the largest share of cancellations across the dataset. Plugging next-day forecast data into the model's severe weather inputs generates a forward-looking risk score up to 24 hours in advance, giving Network Operations Centers a structured, data-driven trigger to activate staffing plans, pre-position spare aircraft, and open rebooking windows before conditions deteriorate.

**Route & Schedule Optimization.** With the model predicting delay severity to within approximately 19 minutes on average, scheduling teams have a quantitative basis for adding targeted buffer time on the highest-risk routes. Route delay rates in this dataset ranged from 2% to nearly 49%, and routes at the high end represent clear, prioritized candidates for schedule adjustment to reduce missed connections and the associated rebooking and compensation costs.

**Passenger Communication & Rebooking Automation.** The regression model's delay estimate in minutes enables personalized, threshold-based rebooking offers to be pushed automatically to passengers at risk of missing a connection before they reach the gate. This reduces inbound customer service volume during disruptions and addresses one of the most visible and costly friction points in the passenger experience.

**Final Recommendation.** The technology is proven, the data infrastructure is in place, and the business case is clear. Deploy the Gradient Boosting model into live operations. Every day without a predictive delay system is a day of avoidable disruptions, reactive decision-making, and preventable customer churn. The question is no longer whether predictive modeling can reduce delays. It is how quickly the organization is willing to act on it.

**Phase 7: Known Constraints and Future Considerations.**

**Model Limitations.**
Despite these successes, we acknowledge certain limitations in the current data scope, such as the absence of Tail-Number maintenance history and mechanical logs, which could further isolate weather-driven delays from aircraft-specific failures. If given more time, we would prioritize the integration of Real-Time Air Traffic Control (ATC) feeds to account for tactical ground stops that occur mid-flight. Furthermore, we would implement smarter data indexing techniques to reduce the processing time required when combining large flight and weather datasets at scale, which would meaningfully improve the speed and efficiency of the pipeline in a production environment.

**Future Considerations**
The immediate priority should be transitioning the model from a historical analysis tool into a live scoring pipeline that generates rolling delay probabilities throughout the day using real-time weather and departure data. In parallel, expanding the training data to include aircraft maintenance logs and airport congestion levels would sharpen the model's ability to distinguish weather-driven disruptions from mechanical ones, improving both accuracy and operational trust. Longer term, evolving the model to track individual aircraft across their full daily flight sequence would unlock

the ability to detect cascading delay chains before they develop, shifting the organization from reactive disruption management to true network-level prevention.

# Appendix

**Exhibit 1:** Open-Meteo JSON Parsing and Weather Feature Engineering. New weather_daily.csv file.

20

```
# import requests
# import time
# import pandas as pd

start_date = "2015-01-01"
end_date   = "2015-12-31"

# Daily variables
daily_vars = [
    "temperature_2m_mean",
    "precipitation_sum",
    "wind_speed_10m_max",
    "wind_gusts_10m_max"
]

#def fetch_with_retry(url, params, max_tries=6):
    wait = 3  # seconds
    for attempt in range(max_tries):
        r = requests.get(url, params=params, timeout=60)
        if r.status_code == 200:
            return r.json()
        if r.status_code == 429:
            time.sleep(wait)
            wait *= 2  # exponential backoff
            continue
        r.raise_for_status()
    raise Exception("Too many 429s - try smaller topN or rerun later")

rows = []
url = "https://archive-api.open-meteo.com/v1/archive"
```

```python
# With topN=100, this works with backoff.
# for r in coords:
    airport = r["AIRPORT"]
    lat = float(r["LAT"])
    lon = float(r["LON"])

    #params = {
        "latitude": lat,
        "longitude": lon,
        "start_date": start_date,
        "end_date": end_date,
        "daily": ",".join(daily_vars),
        "timezone": "auto"
    }

    data = fetch_with_retry(url, params)

    dates = data["daily"]["time"]
    tmean = data["daily"]["temperature_2m_mean"]
    psum  = data["daily"]["precipitation_sum"]
    wsmax = data["daily"]["wind_speed_10m_max"]
    wgmax = data["daily"]["wind_gusts_10m_max"]

    for i, d in enumerate(dates):
        rows.append({
            "AIRPORT": airport,
            "WX_DATE": d,
            "temp_avg": float(tmean[i]) if tmean[i] is not None else None,
            "precip_sum": float(psum[i]) if psum[i] is not None else None,
            "wind_speed_max": float(wsmax[i]) if wsmax[i] is not None else None,
            "wind_gust_max": float(wgmax[i]) if wgmax[i] is not None else None,
        })

    time.sleep(1)  # small pause
```

```python
    time.sleep(1)  # small pause

weather_daily_pd = pd.DataFrame(rows)
weather_daily = spark.createDataFrame(weather_daily_pd).withColumn("WX_DATE", F.to_date("WX_DATE"))

# Flags (including the IMPORTANT severe rule)
weather_daily = (weather_daily
    .withColumn("precip_flag", (F.col("precip_sum") > 0).cast("int"))
    .withColumn("severe_weather_flag",
                ((F.col("precip_sum") >= 10) | (F.col("wind_gust_max") >= 40)).cast("int"))
)

display(weather_daily.limit(5))
print("weather_daily rows:", weather_daily.count())
```

**Exhibit 1 (a):** Share of flights matched with weather data and average severe weather rate.

| rows | share_with_weather | avg_severe_flag |
|---|---|---|
| 5819079 | 0.8219680468335281 | 0.2642528138903081 |

**Exhibit 2:** Delay Rate by Month. EDA.

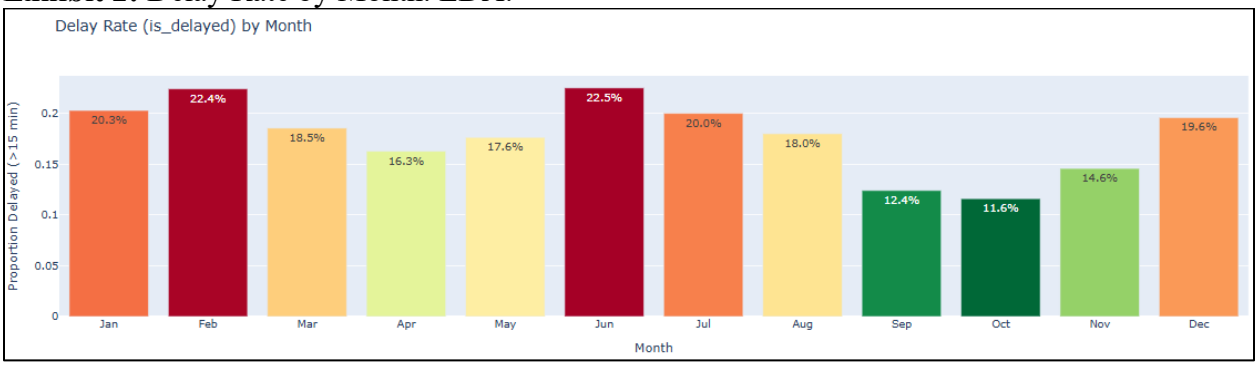

**Exhibit 3:** Delay Rate by Day of Week. EDA.

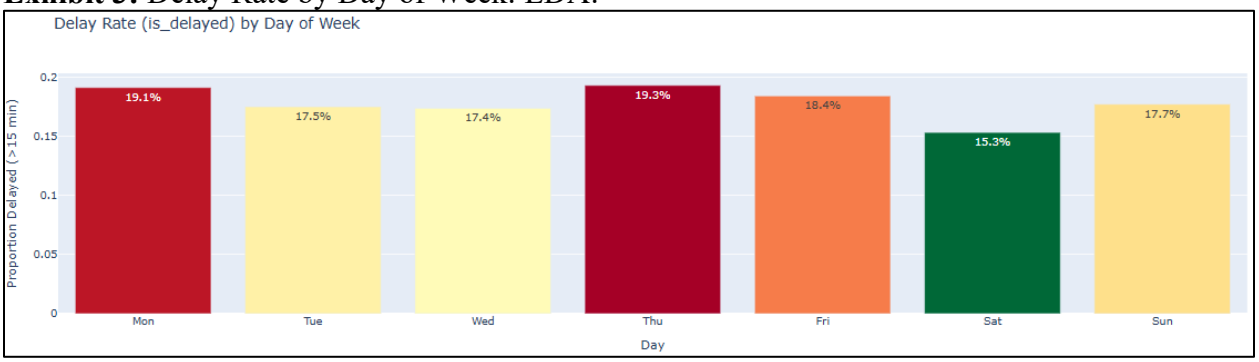


**Exhibit 4:** Mean Arrival Delay by Month. EDA.

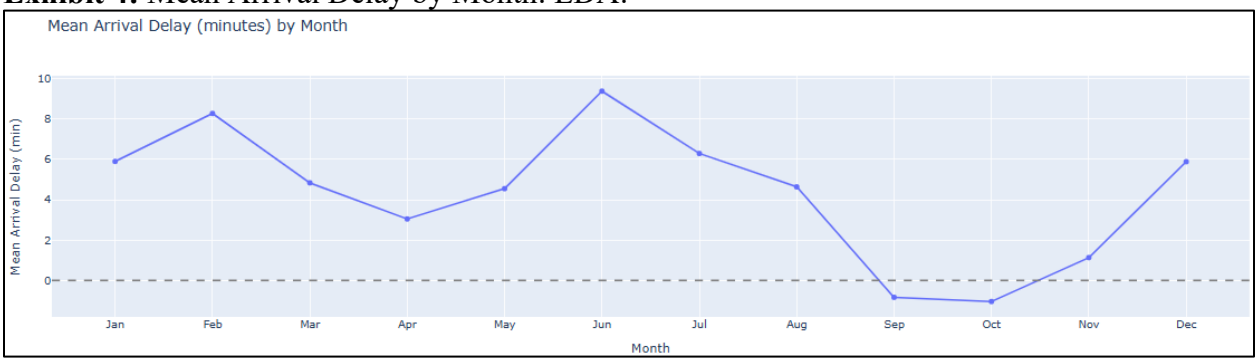


**Exhibit 5:** Weather Flags vs Delay Rate. EDA.

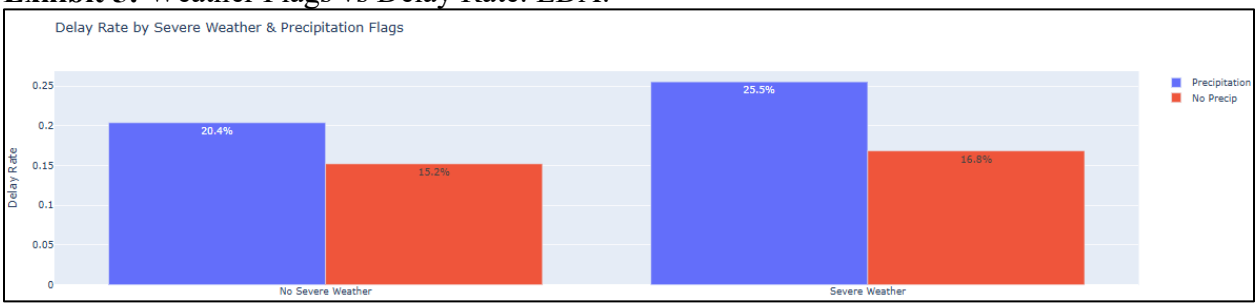


**Exhibit 6:** Airline Summary Table. EDA

Airline summary:

| AIRLINE_NAME | n_flights | delay_rate | mean_arr_delay | cancel_rate |
|---|---|---|---|---|
| Southwest Airlines Co. | 1261855 | 0.18 | 4.375 | 0.013 |
| Delta Air Lines Inc. | 875881 | 0.129 | 0.187 | 0.004 |
| American Airlines Inc. | 725984 | 0.173 | 3.451 | 0.015 |
| Skywest Airlines Inc. | 588353 | 0.177 | 5.846 | 0.017 |
| Atlantic Southeast Airlines | 571977 | 0.184 | 6.585 | 0.027 |
| United Air Lines Inc. | 515723 | 0.196 | 5.432 | 0.013 |
| American Eagle Airlines Inc. | 294632 | 0.199 | 6.458 | 0.051 |
| JetBlue Airways | 267048 | 0.214 | 6.678 | 0.016 |
| US Airways Inc. | 198715 | 0.176 | 3.706 | 0.02 |

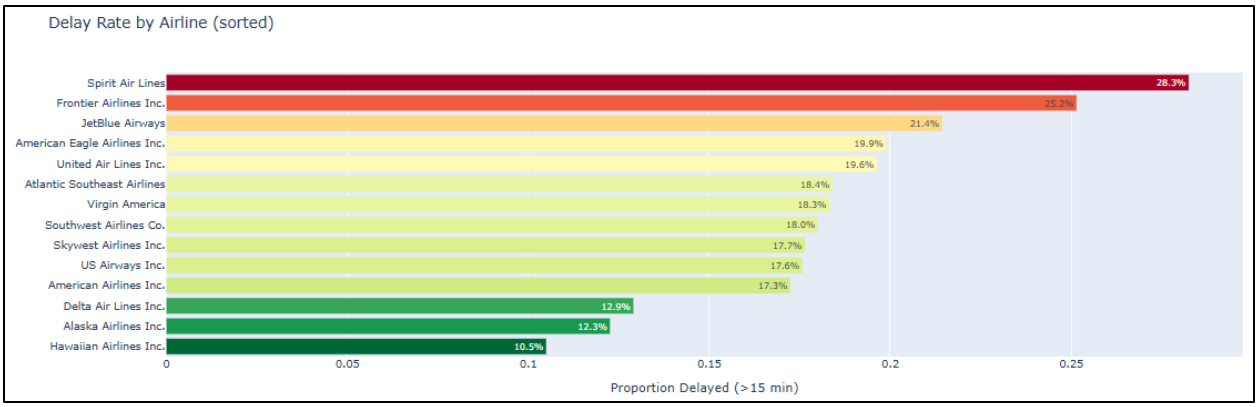


**Exhibit 7:** Top 20 origin airports ranked by flight delay rate (≥5,000 departures). EDA.

**Exhibit 8**: Relationship between precipitation levels and arrival delays (5% sample, 99th percentile cap). EDA.

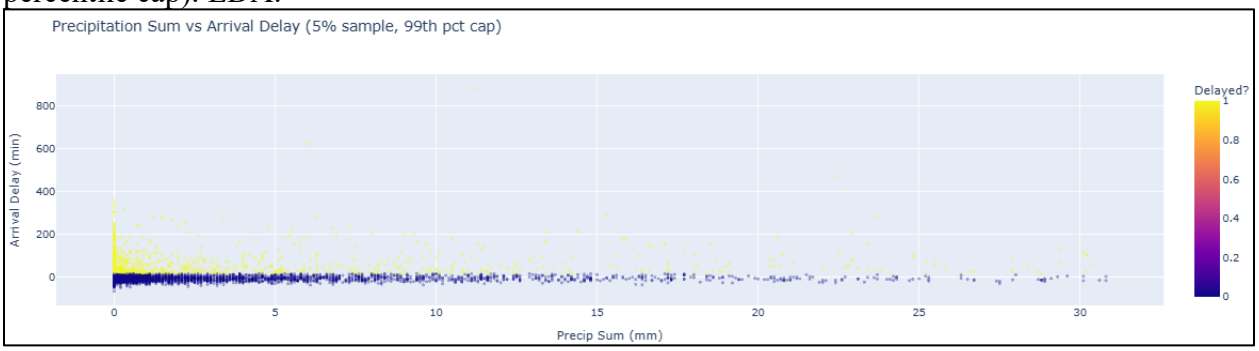


**Exhibit 9**: Relationship between route distance and flight delay rate (bubble size represents number of flights). EDA.

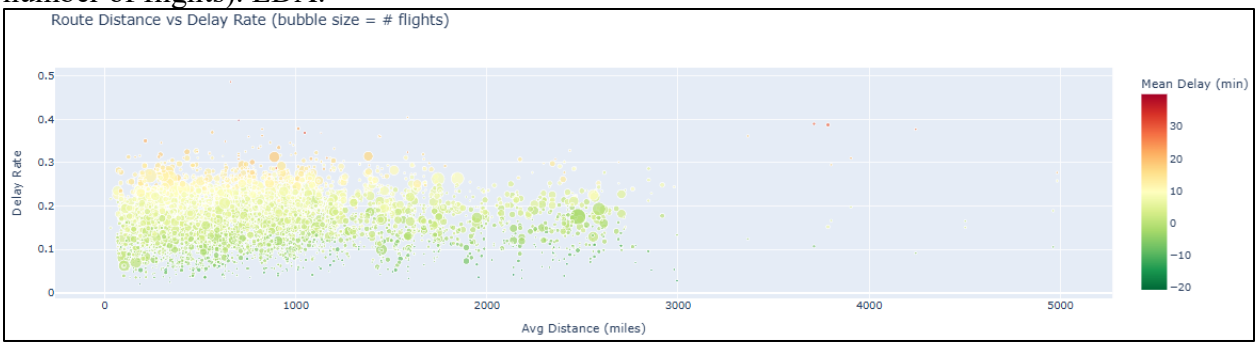

**Exhibit 10:** Preprocessing / Feature Engineering Pipeline (Notebook PREP-1 to PREP-5)

- PREP-1: Cyclical Encoding — Convert SCHEDULED_DEPARTURE, MONTH, and DAY_OF_WEEK into sin/cos pairs to preserve circular structure.
- PREP-2: Weather Median Imputation — Compute medians on the training split only, then apply to both train and test to avoid leakage.
- PREP-3: ARRIVAL_DELAY Transformations — Create ARRIVAL_DELAY_CAPPED (cap at 99th percentile) and ARRIVAL_DELAY_LOG (log1p after cap/shift) so targets are stable for modeling.
- PREP-4: DISTANCE Transform — Apply log1p to distance to reduce right-skew; use LOG_DISTANCE in features.
- PREP-5: Final Feature Set ("Rebuilt Feature Pipeline") — Update numeric feature vector to use sin/cos time features, LOG_DISTANCE, and imputed weather columns; choose regression label (ARRIVAL_DELAY_CAPPED or ARRIVAL_DELAY_LOG) by setting labelCol.

**Exhibit 11:** Descriptive statistics for raw, capped, and log-transformed arrival delay variables.

Descriptive stats comparison:

| | ARRIVAL_DELAY | ARRIVAL_DELAY_CAPPED | ARRIVAL_DELAY_LOG |
|---|---|---|---|
| count | 285318.000 | 285318.000 | 285318.000 |
| mean | 4.347 | 3.403 | 4.484 |
| std | 39.337 | 31.717 | 0.270 |
| min | -73.000 | -73.000 | 2.773 |
| 25% | -13.000 | -13.000 | 4.331 |
| 50% | -5.000 | -5.000 | 4.431 |
| 75% | 8.000 | 8.000 | 4.575 |
| max | 1463.000 | 164.000 | 5.533 |

**Exhibit 12:** Model performance comparison for flight delay classification (AUC-ROC, F1, Precision, Recall).

CLASSIFICATION RESULTS - is_delayed (>15 min)

| | model | AUC-ROC | F1 | Precision | Recall |
|---|---|---|---|---|---|
| 1 | GBT (tuned) | 0.6774 | 0.7494 | 0.7882 | 0.8231 |
| 2 | GBT (untuned) | 0.6774 | 0.7494 | 0.7882 | 0.8231 |
| 3 | Logistic Regression | 0.6655 | 0.7435 | 0.7683 | 0.8216 |
| 4 | Random Forest | 0.6104 | 0.7410 | 0.6748 | 0.8215 |

**Exhibit 13** – Model Comparison Table (Regression)

```
=======================================================
REGRESSION RESULTS - ARRIVAL_DELAY_CAPPED
=======================================================
```

| | model | RMSE | MAE | R2 |
|---|---|---|---|---|
| 1 | GBT (tuned) | 30.5897 | 19.0757 | 0.0743 |
| 2 | GBT (untuned) | 30.5897 | 19.0757 | 0.0743 |
| 3 | Linear Regression | 30.9304 | 19.3896 | 0.0536 |
| 4 | Random Forest | 30.9568 | 19.2949 | 0.0520 |

**Exhibit 14**: MLflow Run History (Regression + Classification).

```
=== MLflow: Regression Run History ===
             run_name     params.model_type        params.target params.maxDepth params.maxIter params.numTrees  metrics.RMSE  metrics.MAE  metrics.R2
        GBT_reg_tuned          GBTRegressor ARRIVAL_DELAY_CAPPED               5             20            None       30.5897      19.0757      0.0743
      GBT_reg_untuned          GBTRegressor ARRIVAL_DELAY_CAPPED               5             20            None       30.5897      19.0757      0.0743
    Random_Forest_reg RandomForestRegressor ARRIVAL_DELAY_CAPPED               5           None              50       30.9568      19.2949      0.0520
Linear_Regression_reg      LinearRegression ARRIVAL_DELAY_CAPPED            None             20            None       30.9304      19.3896      0.0536

=== MLflow: All Runs (Classification + Regression) ===
               run_name    params.task      params.model_type  metrics.AUC-ROC  metrics.F1  metrics.RMSE  metrics.R2
          GBT_reg_tuned     regression           GBTRegressor              NaN         NaN       30.5897      0.0743
        GBT_reg_untuned     regression           GBTRegressor              NaN         NaN       30.5897      0.0743
      Random_Forest_reg     regression  RandomForestRegressor              NaN         NaN       30.9568      0.0520
  Linear_Regression_reg     regression       LinearRegression              NaN         NaN       30.9304      0.0536
          GBT_clf_tuned classification          GBTClassifier           0.6774      0.7494           NaN         NaN
        GBT_clf_untuned classification          GBTClassifier           0.6774      0.7494           NaN         NaN
      Random_Forest_clf classification RandomForestClassifier           0.6104      0.7410           NaN         NaN
Logistic_Regression_clf classification     LogisticRegression           0.6655      0.7435           NaN         NaN
```